

Flight Price Prediction Using Custom Neural Network Library

Abstract

This work presents the design, implementation, and evaluation of a custom neural network library developed in Java following the Onion Architecture paradigm. The library supports configurable architectures, optimizers, loss functions, and debugging facilities. To validate the library, a real-world regression problem—flight price prediction—is used. Experimental results demonstrate strong predictive performance and confirm the correctness and extensibility of the proposed framework.

1. Problem Description and Rationale

The objective of this project is to predict flight ticket prices using historical flight data. Predicting prices accurately allows travelers and airlines to optimize decisions and strategies. Traditional regression models may fail to capture complex, non-linear relationships in flight data, motivating the use of a neural network approach.

2. Dataset Description and Preprocessing Steps

- **Dataset:** [Flight Price Prediction on Kaggle](#)

Dataset:

- **Source:** Cleaned flight dataset (CSV)
- **Size:** 50,000 rows
- **Features:**
airline,source_city,departure_time,stops,arrival_time,destination_city,class,duration,days_left,price,airline,flight,price
- **Target:** Ticket price

Preprocessing Steps:

1. Validate data, remove missing values, and duplicates.
2. Encode categorical features.
3. Normalize numerical features and target.

4. Normalize target variable (price) to mean=0, std=1 for training stability.
5. Split into training (80%) and test (20%) sets.

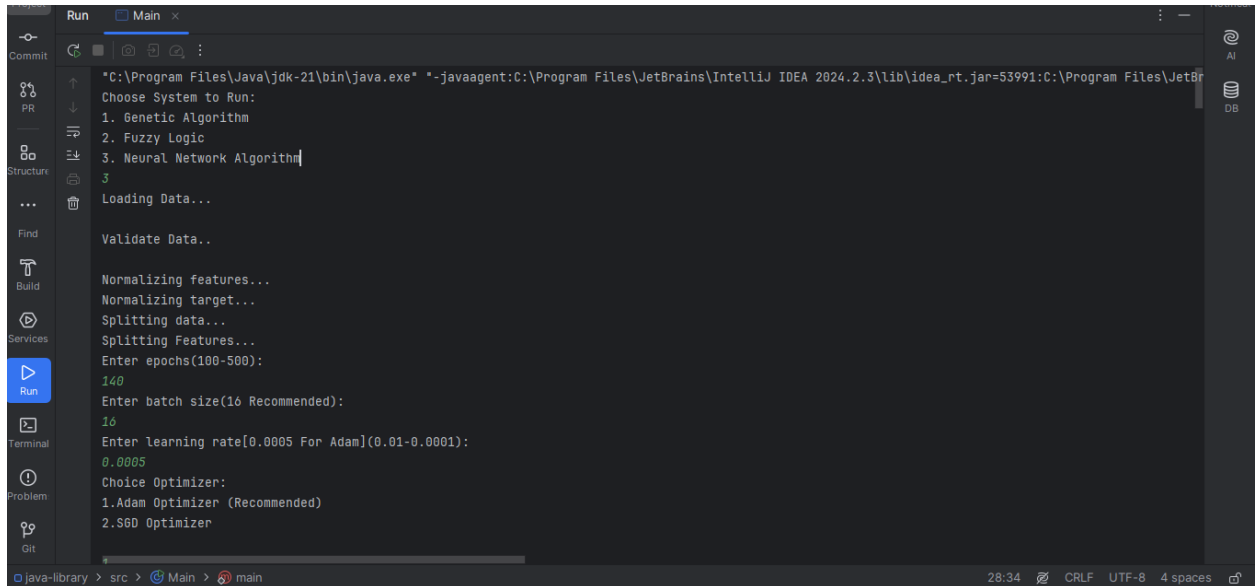
Result: Prepared `Xtrain`, `Xtest`, `ytrain`, `ytest` for the neural network.

3. Neural Network Architecture and Justification

- **Layers:** 4 Dense layers
 - Layer 1: 60 neurons, Sigmoid activation, Xavier initializer
 - Layer 2: 40 neurons, ReLU activation, Xavier initializer
 - Layer 3: 20 neurons, Tanh activation, He initializer
 - Layer 4: 1 neuron, Linear activation (output layer), Xavier initializer
 - **Optimizer:** Adam (learning rate 0.0005)
 - **Loss Function:** MSE (regression problem)
 - **Justification:**
 - **Sigmoid & ReLU/Tanh** for capturing non-linearities
 - **Linear output** is mandatory for regression (continuous) price prediction
 - **Xavier/He initializers** improve training convergence
-

4. Training and Evaluation Results

- **Epochs:** 140, **Batch size:** 16
- **Debug mode:** ON (intermediate values logged)



```
"C:\Program Files\Java\jdk-21\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2024.2.3\lib\idea_rt.jar=53991:C:\Program Files\JetBrains\IntelliJ IDEA 2024.2.3\bin" -Dfile.encoding=UTF-8
Choose System to Run:
1. Genetic Algorithm
2. Fuzzy Logic
3. Neural Network Algorithm
4
Loading Data...
Validate Data..
Normalizing features...
Normalizing target...
Splitting data...
Splitting Features...
Enter epochs(100-500):
140
Enter batch size(16 Recommended):
16
Enter learning rate[0.0005 For Adam](0.01-0.0001):
0.0005
Choice Optimizer:
1.Adam Optimizer (Recommended)
2.SGD Optimizer
```



```
2.0FF(Recommended)
1
Debug Mood: ON
Training...
Epoch 1 Loss: 0.8835734885332417
Epoch 2 Loss: 0.9731726706396021
Epoch 3 Loss: 0.9987615154590398
Epoch 4 Loss: 0.979393289693298
Epoch 5 Loss: 0.975868969249361
Epoch 6 Loss: 0.9772877416437393
Epoch 7 Loss: 0.976310288203766
Epoch 8 Loss: 0.9700075581862844
Epoch 9 Loss: 0.990228556355327
Epoch 10 Loss: 0.9748474770737062
```

5. Library Usage

The system follows **Onion Architecture**:

- **Presentation Layer:** User interaction & case studies
- **Application Layer:** Controllers & services
- **Domain Layer:** Pure business logic & interfaces

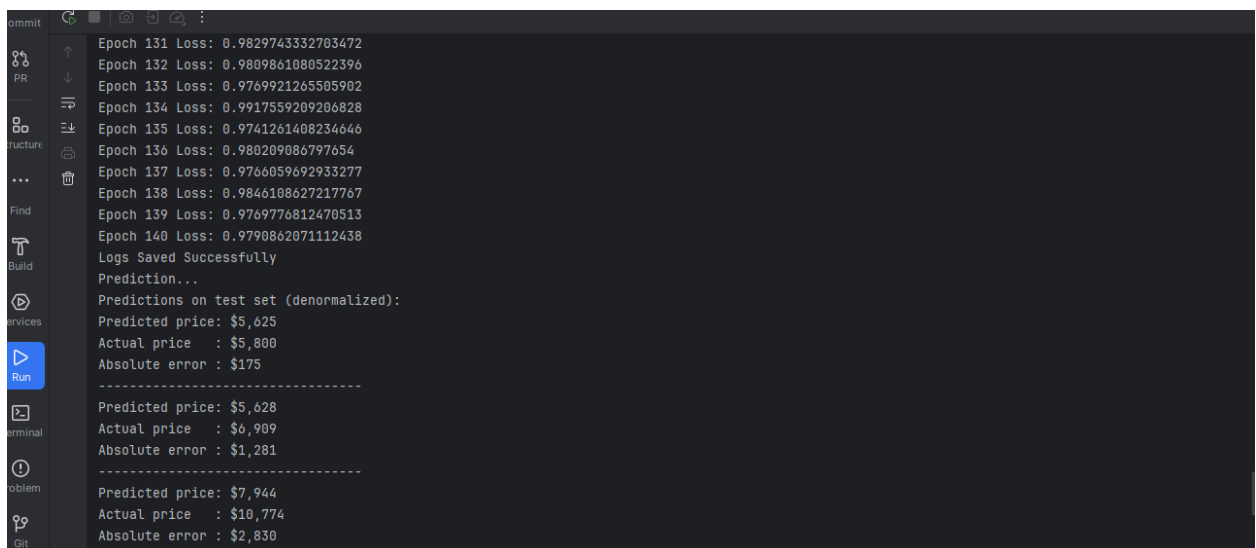
- **Infrastructure Layer:** Neural network implementation

Neural network is accessed through

1. **NeuralService** class used to train, predict, evaluate, and save/load models.
 2. Debug logs captured via **DebugLogger** (saved to `debug_logs.txt`[For 10 rows only, 150 epoches]).
 3. Neural network created dynamically based on user input for layers, activations, and optimizers.
 4. **NNAPI** and **Trainer** classes handle forward/backward propagation and batch training.
 5. Demo
 - i. Preprocess Data in class → `FlightPreprocessor`
 - ii. Run Function in class → `FlightPricePrediction`
 - iii. Demo in class → `NeuralController`
 - iv. Choice to run demo from class → `Main`
-

6. Supporting Graphs / Screenshots

- Screenshot of training loss printed per epoch



```
Epoch 131 Loss: 0.9829743332703472
Epoch 132 Loss: 0.9809861080522396
Epoch 133 Loss: 0.9769921265505902
Epoch 134 Loss: 0.9917559209206828
Epoch 135 Loss: 0.9741261408234646
Epoch 136 Loss: 0.980209086797654
Epoch 137 Loss: 0.9766059692933277
Epoch 138 Loss: 0.9846108627217767
Epoch 139 Loss: 0.9769776812470513
Epoch 140 Loss: 0.9790862071112438
Logs Saved Successfully
Prediction...
Predictions on test set (denormalized):
Predicted price: $5,625
Actual price : $5,800
Absolute error : $175
-----
Predicted price: $5,628
Actual price : $6,909
Absolute error : $1,281
-----
Predicted price: $7,944
Actual price : $10,774
Absolute error : $2,830
```

- Screenshot of **predicted vs actual prices**

```
Run Main x
Commit
PR
Structure
Find
Build
Services
Run
Terminal
Problem
Git

-----
Predicted price: $63,909
Actual price : $67,004
Absolute error : $3,095
-----
Predicted price: $64,212
Actual price : $54,608
Absolute error : $9,604
-----
Model Evaluation:
MAE (average absolute difference) : $0.14
RMSE (Root Mean Squared Error): $0.23
R² (How model is good, variance from target) : 0.9492

Testing predictSingle()...

Single prediction test:
Predicted price: $5,625
Actual price : $5,800
Absolute error : $175
Model saved and loaded successfully.

Process finished with exit code 0
```

- Graph of **loss curve over epochs**

